

React Native Fundamentals

Component

- You can think of components as blueprints. Whatever a function component returns is rendered as a **React element**. React elements let you describe what you want to see on the screen.
- Here the **Hello** component will render a `<Text>` element:

```
const Hello = () => {  
  return <Text>Hello, I am your cat!</Text>;  
};
```

```
import { StatusBar } from 'expo-status-bar';
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const Hello = () => {
  return (
    <View style={styles.container}>
      <Text>Hello RN</Text>
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});

export default Hello
```

JSX (JavaScript Syntax Extension)

- React and React Native use JSX, a syntax that lets you write elements inside JavaScript like so: `<Text>Hello, I am JSX!</Text>`.
- Because JSX is JavaScript, you can use variables inside it.

```
const Hello = () => {  
  const name = 'JSX'  
  return (  
    <View style={styles.container}>  
      <Text>Hello, I am {name}</Text>  
    </View>  
  );  
}
```

```
import { StatusBar } from 'expo-status-bar';
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const getFullName = (firstName, surName) => {
  return firstName + ' ' + surName
}

const Teacher = () => {
  const name = 'Chakkrit'
  const surname = 'Preuksakarn'
  return (
    <View style={styles.container}>
      <Text>Hello {getFullName(name, surname)}</Text>
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});

export default Teacher
```

Custom Components

- React lets you nest Core components inside each other to create new components. These nestable, reusable components are at the heart of the React paradigm.
- For example, you can nest **Text** inside a **View** below, and React Native will render them together:

```
const Hello = () => {  
  return (  
    <View style={styles.container}>  
      <Text>Hello, RN</Text>  
      <Text>How are you?</Text>  
    </View>  
  );  
}
```

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const MobileApp = () => {
  return(
    <View>
      <Text>Mobile Application Development</Text>
    </View>
  );
}

const Subject = () => {
  return(
    <View style={styles.container}>
      <Text>Welcome!</Text>
      <MobileApp/>
      <MobileApp/>
      <MobileApp/>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});

export default Subject
```

Props

- Props is short for “properties”. Props let you customize React components. For example, here you pass each `<MobileApp>` a different lesson for MobileApp to render:
- You can build many things with props and the Core Components `Text`, `Image`, and `View`! But to build something interactive, you’ll need state.


```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const MobileApp = (props) => {
  return(
    <View>
      <Text>Mobile Application Development: {props.lesson}</Text>
    </View>
  );
}

const Subject = () => {
  return(
    <View style={styles.container}>
      <Text>Welcome!</Text>
      <MobileApp lesson='Components' />
      <MobileApp lesson='JSX' />
      <MobileApp lesson='Props' />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});

export default Subject
```

State

- While you can think of props as arguments you use to configure how components render, **state** is like a component's personal data storage. State is useful for handling data that changes over time or that comes from user interaction. State gives your components memory!

```
import React, {useState} from 'react';
import { StyleSheet, Text, View, Button } from 'react-native';
```

```
//State
const MobileApp = (props) => {
  const [isLearning, setLearning] = useState(false)
  return(
    <View>
      <Text>Learning: {props.lesson} is {isLearning ? 'done' : 'waiting'}.</Text>
      <Button
        onPress={() => {
          setLearning(true)
        }}
        disabled={isLearning}
        title={isLearning ? 'Done!' : 'OK'}
      />
    </View>
  );
}
```

```
const Subject = () => {
  return(
    <View style={styles.container}>
      <MobileApp lesson='JSX' />
      <MobileApp lesson='Props' />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

```
export default Subject
```